

Orbit Wars — RL Formulation and GRPO Training Math

Native C++/LibTorch stack

June 3, 2026

This document specifies the policy, the reward, and the *Group Relative Policy Optimization* (GRPO) update used by the native trainer. Symbols map to `native/rl/config.hpp` and `native/rl/model/`.

Contents

1	Problem setup	1
2	Observation encoding	2
3	Policy network π_θ	2
4	Action distribution	3
5	Reward and return	3
6	GRPO update	4
6.1	Grouped sampling	4
6.2	Group-relative advantage (no critic)	4
6.3	Clipped policy surrogate	5
6.4	KL anchor to a reference policy	5
6.5	Total objective	5
6.6	Optimization	5
7	Self-play league	6
8	Symbols $\leftrightarrow$ config.hpp	6
A	Appendix: the PPO/GAE method being replaced	6

1 Problem setup

Orbit Wars is an N -player symmetric Markov game with $N \in \{2, 4\}$. The engine sets $N = |\text{state}|$ (`num_agents`) and assigns N home planets from a 4-fold-symmetric quadrant group ($N = 2$: two opposite quadrants; $N = 4$: all four). Each turn all players *simultaneously* issue a set of fleet *launches*; the deterministic engine then moves fleets, resolves collisions/combat, rotates planets, produces ships, and (on scheduled steps) spawns comets. Scoring is a **free-for-all**: at termination player i 's score is its total ship count ships_i (planets + fleets), and the seat(s) with the maximum

score receive reward +1, all others −1 (so $N = 4$ is winner-take-all among four individuals — not teams).

Training scope of this document. We currently train the **two-player special case** ($N = 2$: ego $p = 0$ against one opponent in the opposite quadrant). The native world generator places exactly two homes; the shaped reward (Section 5) uses the zero-sum margin $\text{ships}_0 - \text{ships}_1$; and each group plays a single opponent (scripted, or a frozen snapshot, Section 7). The policy (Section 3) and the GRPO update (Section 6) are *N-agnostic*; lifting to 4-player FFA requires only (i) 4-home world generation, (ii) replacing the zero-sum margin with the FFA return (max-score seat +1 else −1, or a rank-based shaping), and (iii) seating ≥ 1 extra opponents per game. This 2p→4p gap is an open modeling item, revisited in Section 5.

Factored action space. A turn’s action is *factored over the ego player’s planets* (“entities”). The observation exposes up to E entity slots ($E = 64$). Each owned planet e independently chooses a class

$$a_e \in \{0, 1, \dots, A - 1\}, \quad A = 1 + C |\mathcal{F}|, \quad (1)$$

where class 0 is *no-op*, $\mathcal{F} = \{0.25, 0.5, 0.75, 1.0\}$ is the set of ship-fraction choices, and C is the number of target choices: $C = E$ in *target mode* (each class ≥ 1 names a target entity t and a fraction f , with aim angle computed analytically to hit t), or $C = \text{angle_bins}$ in *angle mode* (class names an angle bin and a fraction). The class index for a launch is

$$\text{class}(t, f) = 1 + t |\mathcal{F}| + f. \quad (2)$$

2 Observation encoding

For ego player p and state s , the encoder (`native/core/encode.hpp`) emits

- entity features $X \in \mathbb{R}^{E \times F}$ ($F = 15$): per planet/fleet geometry, ownership (ego/ally/enemy/neutral one-hots), ships, production, comet flags;
- entity mask $m \in \{0, 1\}^E$: $m_e = 1$ iff slot e is a real entity;
- action mask $u \in \{0, 1\}^E$: $u_e = 1$ iff e is an ego-owned planet with > 0 ships (i.e. legally able to launch);
- globals $g \in \mathbb{R}^G$ ($G = 10$): step/phase, board-wide ship and production margins, counts, angular velocity, etc.

3 Policy network π_θ

The network (`native/rl/model/policy_net.hpp`) maps (X, m, u, g) to per-entity class logits $Z \in \mathbb{R}^{E \times A}$. It has **no value head**; GRPO needs no critic. Let $d = 128$ (**hidden**).

Entity encoder. A shared two-layer MLP over each entity row:

$$h_e = W_2 \text{ReLU}(W_1 x_e + b_1) + b_2 \in \mathbb{R}^d, \quad e = 1, \dots, E. \quad (3)$$

Context encoder. A mask-aware mean pool followed by a two-layer MLP on the pooled tokens concatenated with the globals:

$$\bar{h} = \frac{\sum_e m_e h_e}{\max(\sum_e m_e, 1)}, \quad c = W_4 \text{ReLU}(W_3 [\bar{h}; g] + b_3) + b_4 \in \mathbb{R}^d. \quad (4)$$

Actor head. Two interchangeable heads produce the raw logits $\tilde{Z}$:

Angle mode: an MLP on each entity token concatenated with the context,

$$\tilde{z}_e = W_6 \text{ReLU}(W_5 [h_e; c] + b_5) + b_6 \in \mathbb{R}^A. \quad (5)$$

Target mode (pointer actor): for each launcher e and fraction f a query $q_{e,f} \in \mathbb{R}^d$ (from $W_q[h_e; c]$ reshaped to $|\mathcal{F}| \times d$), for each target t a key $k_t = W_k h_t$, and a per-launcher no-op logit $n_e = w_n^\top [h_e; c]$:

$$\text{score}(e, t, f) = \frac{\langle q_{e,f}, k_t \rangle}{\sqrt{d}}, \quad \tilde{z}_{e,0} = n_e, \quad \tilde{z}_{e,\text{class}(t,f)} = \text{score}(e, t, f). \quad (6)$$

The score depends on *both* launcher and target encodings, so the head can rank targets by geometry (the property a plain MLP actor lacks).

Legality masking. Illegal classes are set to $-\infty$ (numerically -10^9). Class 0 (no-op) is always legal. For a launch class (t, f) ,

$$Z_{e,\text{class}(t,f)} = \begin{cases} \tilde{z}_{e,\text{class}(t,f)} & \text{if } u_e = 1 \wedge m_t = 1 \wedge t \neq e \quad (\text{target mode}), \\ \tilde{z}_{e,\text{class}(t,f)} & \text{if } u_e = 1 \quad (\text{angle mode}), \\ -\infty & \text{otherwise.} \end{cases} \quad (7)$$

4 Action distribution

Each entity is an independent masked categorical; a turn’s joint action is their product over *real* entities (`native/rl/model/distribution.hpp`):

$$\pi_\theta(a_e | s) = \text{softmax}(Z_e)_{a_e}, \quad \pi_\theta(a | s) = \prod_{e: m_e=1} \pi_\theta(a_e | s). \quad (8)$$

Hence the joint log-probability and entropy are entity sums:

$$\log \pi_\theta(a | s) = \sum_e m_e \log \text{softmax}(Z_e)_{a_e}, \quad (9)$$

$$\mathcal{H}[\pi_\theta(\cdot | s)] = \sum_e m_e \left(- \sum_{a'} \text{softmax}(Z_e)_{a'} \log \text{softmax}(Z_e)_{a'} \right). \quad (10)$$

Sampling draws $a_e \sim \text{Categorical}(\text{softmax}(Z_e))$ per entity (greedy uses $\arg \max_a Z_{e,a}$ at evaluation).

5 Reward and return

Shaped potential. Let $\text{ships}_p(s)$ and $\text{prod}_p(s)$ be player p ’s total ships (planets + fleets) and total planetary production in s . Define the production-aware potential (`native/rl/reward; prod_weight= $w_\Phi$`)

$$\Phi(s) = (\text{ships}_0(s) - \text{ships}_1(s)) + w_\Phi (\text{prod}_0(s) - \text{prod}_1(s)). \quad (11)$$

Including production makes *capturing a planet* immediately rewarding, counteracting the short-term ship cost of expansion that made a ship-only reward collapse to passivity.

Per-step shaped reward. With scale σ (`reward_scale`) and clip κ (`reward_clip`),

$$r_t^{\text{sh}} = \text{clip}\left(\frac{\Phi(s_{t+1}) - \Phi(s_t)}{\sigma}, -\kappa, \kappa\right). \quad (12)$$

Outcome. At the terminal step T_i of trajectory i , the game is scored by total ships; let $o_i = +1$ if ego wins, -1 if it loses, 0 on a draw.

GRPO return. GRPO scores a *whole trajectory* by a scalar return blending the discounted shaped reward and the outcome (`gamma` γ , `weights` $w_d = \text{dense_weight}$, $w_o = \text{outcome_weight}$):

$$R_i = w_d \sum_{t=0}^{T_i-1} \gamma^t r_{i,t}^{\text{sh}} + w_o o_i \quad (13)$$

Setting $w_d = 0$ recovers *pure-outcome* GRPO; $\gamma = 1$ is the undiscounted Monte-Carlo return (the default).

Note — 2p vs. 4p objective. The margin potential Φ and the outcome o_i are the two-player ($N = 2$) zero-sum forms. In $N = 4$ free-for-all the objective is *ordinal* — be the maximum of four — which is not zero-sum: e.g. Φ would become $\text{ships}_0 - \max_{k \neq 0} \text{ships}_k$ (margin over the *strongest* rival, or a soft-max thereof) and o_i the FFA win/lose of Section 1. The GRPO machinery (advantage, surrogate, KL) is unchanged — only R_i in Eq. (13) is redefined.

6 GRPO update

GRPO replaces the value baseline of actor–critic methods with a *group-relative* baseline: the policy plays a group of trajectories from the *same* start against the *same* opponent, and each trajectory’s return is centered against its group.

6.1 Grouped sampling

For iteration we draw N groups (`num_groups`); group j fixes an initial state $s_0^{(j)}$ (a world) and an opponent, and rolls out G trajectories (`group_size`) under the current behavior policy π_{old} (stochastic). The only within-group variation is the policy’s own action sampling, so the group mean is a valid, low-variance baseline. The env count is $N \cdot G$ (`num_envs`). For every transition we store (X, m, u, g) , the action $a_{i,t}$, the behavior log-prob $\log \pi_{\text{old}}(a_{i,t} \mid s_{i,t})$, and the reference log-prob $\log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t})$ (Section 6.4).

6.2 Group-relative advantage (no critic)

Let $\mathcal{G}_j$ be the trajectories of group j , with mean and standard deviation

$$\mu_j = \frac{1}{G} \sum_{i \in \mathcal{G}_j} R_i, \quad \varsigma_j = \sqrt{\frac{1}{G} \sum_{i \in \mathcal{G}_j} (R_i - \mu_j)^2}. \quad (14)$$

The advantage of trajectory i (group $j = j(i)$), *broadcast to all of its timesteps*, is

$$\hat{A}_i = \begin{cases} \frac{R_i - \mu_j}{\varsigma_j + \varepsilon}, & \text{whiten_advantage} = \text{true}, \\ R_i - \mu_j, & \text{otherwise}, \end{cases} \quad (15)$$

with floor $\varepsilon = \text{adv_eps}$. There is no learned $V(s)$ and no temporal-difference or GAE term — the entire baseline is Eq. (15).

6.3 Clipped policy surrogate

With the per-step importance ratio

$$\rho_{i,t}(\theta) = \exp\left(\log \pi_\theta(a_{i,t} \mid s_{i,t}) - \log \pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})\right), \quad (16)$$

(each log-prob the entity sum of Eq. (9)), the surrogate is the PPO clip objective

$$\mathcal{L}^{\text{clip}}(\theta) = - \frac{1}{\sum_i T_i} \sum_{i,t} \min\left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i\right), \quad (17)$$

with clip range $\epsilon = \text{clip}$.

6.4 KL anchor to a reference policy

A reference policy π_{ref} (the fixed behavior-cloning checkpoint) anchors training, which counteracts the empirically observed drift of plain PPO away from the strong BC behavior. We penalize the unbiased, non-negative k_3 estimator of $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$. With $\Delta_{i,t} = \log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t}) - \log \pi_\theta(a_{i,t} \mid s_{i,t})$,

$$\widehat{\text{KL}}_{i,t} = \exp(\Delta_{i,t}) - \Delta_{i,t} - 1 \quad (\geq 0), \quad \mathcal{L}^{\text{KL}}(\theta) = \frac{1}{\sum_i T_i} \sum_{i,t} \widehat{\text{KL}}_{i,t}. \quad (18)$$

6.5 Total objective

Combining the surrogate, the KL anchor (weight $\beta = \text{kl_beta}$) and an entropy bonus (weight $c_{\mathcal{H}} = \text{ent_coef}$, from Eq. (10)):

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{clip}}(\theta) + \beta \mathcal{L}^{\text{KL}}(\theta) - c_{\mathcal{H}} \overline{\mathcal{H}}[\pi_\theta] \quad (19)$$

where $\overline{\mathcal{H}}$ is the mean entropy over transitions.

6.6 Optimization

For each iteration: collect the grouped trajectories under $\pi_{\theta_{\text{old}}} = \pi_\theta$; compute returns (13) and advantages (15); then for `update_epochs` passes over `minibatches` shuffled minibatches, take Adam steps on (19) with learning rate $\eta = \text{lr}$, $\epsilon_{\text{Adam}} = \text{adam_eps}$, and global gradient-norm clip `max_grad_norm`. Advantages are computed once per iteration (off $\pi_{\theta_{\text{old}}}$); the ratio (17) and KL (18) are recomputed each minibatch under the live π_θ .

7 Self-play league

Once training has reached `start_step`, the opponent for each group is sampled from a mix of `{random, starter, league}` with weights `(mix_random, mix_starter, mix_pool)`. The league is a bounded set (`pool_capacity`) of *historical* frozen snapshots — not a single latest self — which avoids the cycling/forgetting of single-snapshot self-play. Promotion is *win-gated*: a new snapshot is admitted only when the current policy beats the pool with win rate $\geq$ `promote_winrate`, so the policy never trains against a regression of itself.

8 Symbols $\leftrightarrow$ config.hpp

Symbol	config.hpp field	Default
d	<code>ModelConfig::hidden</code>	128
E	<code>ModelConfig::max_entities</code>	64
$\mathcal{F}$	<code>ModelConfig::fractions</code>	<code>{.25, .5, .75, 1}</code>
G, N	<code>GrpoConfig::group_size, num_groups</code>	8, 64
ϵ	<code>GrpoConfig::clip</code>	0.2
β	<code>GrpoConfig::kl_beta</code>	0.04
$c_{\mathcal{H}}$	<code>GrpoConfig::ent_coef</code>	0.01
ε	<code>GrpoConfig::adv_eps</code>	10^{-4}
w_d, w_o	<code>GrpoConfig::dense_weight, outcome_weight</code>	1, 1
γ	<code>GrpoConfig::gamma</code>	1.0
w_{Φ}	<code>RolloutConfig::prod_weight</code>	20
σ, κ	<code>RolloutConfig::reward_scale, reward_clip</code>	50, 5
η	<code>OptimConfig::lr</code>	3×10^{-4}

A Appendix: the PPO/GAE method being replaced

The previous trainer (`native/rl/trainer.hpp`) used actor-critic PPO with a value head V_{ϕ} and Generalized Advantage Estimation. For completeness, with TD residual $\delta_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)$,

$$\hat{A}_t^{\text{GAE}} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \quad \mathcal{L}^V = \frac{1}{2} (V_{\phi}(s_t) - \hat{R}_t)^2, \quad \hat{R}_t = \hat{A}_t^{\text{GAE}} + V_{\phi}(s_t), \quad (20)$$

and total loss $\mathcal{L}^{\text{clip}} + c_V \mathcal{L}^V - c_{\mathcal{H}} \overline{\mathcal{H}}$. GRPO removes V_{ϕ} (and thus λ , c_V , and value warmup) entirely, replacing $\hat{A}_t^{\text{GAE}}$ with the group-relative $\hat{A}_i$ of Eq. (15).